PYTHON & QUANTUM EXPERIMENTS VIVA BANK

Detailed questions with easy-to-say answers

15 topic groups | code-specific questions | worked traces | edge cases | complexity notes | mock viva

How to practise: Try the quick questions first. Then rehearse the expanded answers aloud. For each trace, say what the variables or arrays contain, what each operation does, and why the final result follows. Prompts marked **[General knowledge]** extend beyond the supplied programs.

Based on the supplied cqp.md transcription from 15 PDFs. The guide preserves the source correction notes and does not claim the programs were executed.

Contents

Experiment 1 - Python Basics and Arithmetic 3

Experiment 2 - Conditional Statements 5

Experiment 8 - Loops and Number Problems 7

Experiment 9 - Functions and Arguments 9

Experiment 10 - Recursive Functions 11

Experiment 11 - Core List Operations 13

Experiments 11/12 - Advanced Lists and Sets 15

Experiment 13 - Tuple Operations 17

Experiment 14 - Dictionaries 19

Experiment 15 - Strings and Text Processing 21

Experiment 16 - File Operations 23

Experiment 4 - NumPy Linear Algebra 25

Experiment 5 - Quantum-State Vectors 27

Experiment 6 - Quantum-State Transformations 29

Experiment 7 - Identity, Kronecker Product, and Unitarity 31

Python language examples are grouped separately from NumPy linear algebra and quantum-state calculations.

Experiment 1 - Python Basics and Arithmetic

Program focus: Print, swap, literals, input, average, circle measurements, simple interest, and compound amount.

Quick viva questions

Q1. What does print() do?

Easy answer. It writes values or text to the output stream; multiple arguments are separated by spaces by default.

Q2. How does A, B = B, A swap values?

Easy answer. Python evaluates the right side first as a tuple of the old values, then unpacks them into A and B.

Q3. What type does input() return?

Easy answer. A string. Convert it with int() or float() before doing numeric calculations.

Q4. Why use float() for interest inputs?

Easy answer. It allows amounts, rates, and times with fractional parts.

Q5. What does ** mean in the compound amount formula?

Easy answer. Exponentiation. The amount is P(1+R/100)^T.

Q6. How is simple interest calculated?

Easy answer. P*R*T/100, where R is an annual percent rate and T is time in years.

Q7. Dry run the average of 20, 30, and 40.

Easy answer. The total is 90, divided by 3, so Average is 30.0 because / performs true division.

Q8. What are area and circumference for radius 20 and pi=3.14?

Easy answer. Area=1256; circumference=125.6.

Q9. why choose descriptive variable names? [General knowledge]

Easy answer. They make the purpose of stored values easier to understand and reduce mistakes.

Experiment 1 - Python Basics and Arithmetic - expanded answers

Practice question 1. Explain the swap and input exercises.

Suggested spoken answer. The swap uses simultaneous assignment: Python first evaluates the old values of A and B, then assigns them in reversed order. The input exercise receives text from the user, so numeric input must be converted before arithmetic.

Practice question 2. Compare simple and compound interest in this source.

Suggested spoken answer. Simple interest is calculated only from the original principal, so the interest is P times R times T divided by 100. Compound amount repeatedly applies the rate, represented here by raising (1+R/100) to T; compound interest is the final amount minus the original principal.

Experiment 2 - Conditional Statements

Program focus: Largest of three integers, parity, sign, leap year, and pass/fail.

Quick viva questions

Q1. How does an if/elif/else chain work?

Easy answer. Python tests conditions in order and executes only the first matching branch.

Q2. Why use >= for largest number?

Easy answer. It handles ties; if A equals another maximum it may still be selected.

Q3. How does the even/odd check work?

Easy answer. An integer is even when Num%2 equals zero; otherwise it is odd.

Q4. How does the sign check handle zero?

Easy answer. Zero satisfies neither >0 nor <0, so the else branch identifies it.

Q5. State the leap-year rule in the corrected code.

Easy answer. A year is leap if divisible by 400, or divisible by 4 but not by 100.

Q6. Why is 1900 not leap but 2000 is?

Easy answer. 1900 is divisible by 100 but not 400; 2000 is divisible by 400.

Q7. What does the pass/fail threshold mean?

Easy answer. The program prints Pass for marks at least 35; otherwise it prints Fail.

Q8. What if inputs are not valid integers?

Easy answer. int(input(...)) raises ValueError; this code does not catch it.

Q9. what is a Boolean expression? [General knowledge]

Easy answer. An expression that evaluates to True or False and controls a branch.

Experiment 2 - Conditional Statements - expanded answers

Practice question 1. Explain the leap-year condition.

Suggested spoken answer. The rule handles century years specially. A year divisible by 400 is leap; otherwise it must be divisible by 4 and not divisible by 100. That is why 2000 is leap but 1900 is not.

Practice question 2. How does the largest-of-three logic work with ties?

Suggested spoken answer. It first checks whether A is at least as large as both other values. If not, it checks whether B is at least C; otherwise C is largest. The non-strict comparisons let tied maximum values be handled without missing a branch.

Experiment 8 - Loops and Number Problems

Program focus: Loops print sequences/tables, sum values, compute factorial, count/reverse/sum digits, detect palindromes and primes; includes grade and progressive tax examples.

Quick viva questions

Q1. What is the difference between for and while here?

Easy answer. for iterates over a known range; while repeats as long as a condition remains true and the loop body updates state.

Q2. Why is range(1,N+1) used?

Easy answer. The stop value is excluded, so N+1 is needed to print N.

Q3. How does the factorial loop work?

Easy answer. It multiplies Fact by each integer from 1 through Num; 0! is 1 because the loop runs zero times and Fact starts at 1.

Q4. How does digit counting handle zero?

Easy answer. It starts Count at 1 when Num is zero, because the digit loop would otherwise run zero times.

Q5. How does number reversal preserve a negative sign?

Easy answer. It stores Sign, reverses the absolute value, then multiplies the result by Sign.

Q6. Why is every negative integer declared non-palindrome?

Easy answer. That is the explicit policy in this code; the minus sign is not treated as a digit.

Q7. How does sum-of-digits extract a digit?

Easy answer. Num%10 gives the last decimal digit; Num//10 removes it.

Q8. Why does primality test use i*i <= n?

Easy answer. If n has a factor greater than sqrt(n), it must also have a matching factor below sqrt(n); testing past the square root is unnecessary.

Q9. What is the 6k±1 prime optimization?

Easy answer. After checking 2 and 3, possible prime divisors greater than 3 lie in the forms 6k-1 or 6k+1; test both while their square is <=n.

Q10. Why does the range-prime loop use int(n0.5)+1?**

Easy answer. The upper bound in range is exclusive, so +1 includes the integer square root.

Q11. Explain the progressive tax formula.

Easy answer. Each rate applies only to the income inside its bracket; earlier bracket taxes are added as fixed amounts for higher brackets.

Q12. What limitation exists in the grade/tax examples?

Easy answer. They encode one particular grading/tax schedule and do not validate negative values or policy changes.

Q13. what causes an infinite while loop? [General knowledge]

Easy answer. A loop condition never becomes false, often because the variable that should change is not updated.

Experiment 8 - Loops and Number Problems - expanded answers

Practice question 1. Trace digit reversal for -120.

Suggested spoken answer. The sign is saved as -1. The absolute digits build Rev as 0, then 2, then 21; applying the sign gives -21, so leading zeroes disappear.

Practice question 2. Explain the fast primality check.

Suggested spoken answer. It rejects numbers at most one, accepts two and three, then rejects any even number or multiple of three. It tests candidate divisors in pairs 6k-1 and 6k+1 only while i squared is at most n. This reduces checks compared with testing every integer divisor.

Practice question 3. Explain how the tax brackets avoid taxing all income at the highest rate.

Suggested spoken answer. For each bracket, the code applies that bracket's rate only to the portion above the previous threshold. For income over one million, for example, it adds the tax already accumulated through one million, then applies 30 percent only to the excess.

Experiment 9 - Functions and Arguments

Program focus: Functions demonstrate print-only, return values, factorial, defaults, *args, **kwargs, and multiple returned values.

Quick viva questions

Q1. What is the difference between print and return?

Easy answer. print displays a value; return sends a value back to the caller for further use.

Q2. What does *args collect?

Easy answer. It gathers extra positional arguments into a tuple.

Q3. What does **kwargs collect?

Easy answer. It gathers extra named arguments into a dictionary.

Q4. How do default arguments work in simple_interest?

Easy answer. If the caller omits rate and time, they use 5 and 1; P is still required.

Q5. How are multiple values returned?

Easy answer. Python packages them into a tuple, which can be unpacked into S, Avg, and Mx.

Q6. What is a local variable?

Easy answer. A name assigned inside a function, such as Fact, is local unless declared otherwise.

Q7. What happens if factorial receives a negative number?

Easy answer. This function itself would return 1 because range(1,negative+1) is empty; the main code prevents that call.

Q8. What does check_even_odd return for zero?

Easy answer. Even, because zero modulo two equals zero.

Q9. What is the average of an empty list in calculate?

Easy answer. It raises ZeroDivisionError at Total/len(values); source calls it only with five values.

Q10. what is a docstring? [General knowledge]

Easy answer. A string placed first in a function/module/class body to document it and accessible through __doc__.

Experiment 9 - Functions and Arguments - expanded answers

Practice question 1. Explain *args and **kwargs with the examples.

Suggested spoken answer. The *args function accepts any number of positional values and receives them as a tuple; calculate_sum adds that tuple. The **kwargs function accepts named values and receives them as a dictionary, then loops through each key and value.

Practice question 2. Explain the calculate function and tuple unpacking.

Suggested spoken answer. calculate receives a list, computes the total, average, and maximum, then returns all three. Python returns them together as a tuple, and the assignment S, Avg, Mx = calculate(Numbers) unpacks the tuple into three names.

Practice question 3. Why does the main code check for a negative factorial input?

Suggested spoken answer. The factorial function's loop would perform no multiplications for a negative value and leave the initial result as one, which is mathematically misleading. The main code rejects negative input before calling the function.

Experiment 10 - Recursive Functions

Program focus: Recursive Fibonacci, factorial, and sum of digits.

Quick viva questions

Q1. What is recursion?

Easy answer. A function solving a problem by calling itself on smaller inputs until reaching a base case.

Q2. What are Fibonacci base cases here?

Easy answer. fibonacci(0)=0 and fibonacci(1)=1 because n<=1 returns n.

Q3. What is the factorial base case?

Easy answer. factorial_rec(0) and factorial_rec(1) return 1.

Q4. Why must recursion have a base case?

Easy answer. It stops further calls; without one, calls continue until recursion depth is exceeded.

Q5. How does recursive digit sum reduce the input?

Easy answer. It adds n%10 to the recursive result for n//10.

Q6. What happens for input zero in sum_digits_rec?

Easy answer. It returns zero immediately from the base case.

Q7. What is naive Fibonacci time complexity?

Easy answer. Exponential, approximately O(phi^n), because subproblems are recomputed many times.

Q8. What is recursive factorial complexity?

Easy answer. O(n) calls and O(n) call-stack space.

Q9. Why take abs before sum_digits_rec?

Easy answer. It avoids a negative input whose floor-division sequence would not approach zero in the intended way.

Q10. how improve Fibonacci? [General knowledge]

Easy answer. Memoization or iteration computes each term once, reducing time to O(n).

Experiment 10 - Recursive Functions - expanded answers

Practice question 1. Explain recursive factorial.

Suggested spoken answer. For n greater than one, factorial_rec returns n multiplied by factorial_rec(n-1). The calls continue until zero or one, where the function returns one. Then the pending multiplications unwind; for example 4! becomes 4*3*2*1=24.

Practice question 2. Explain why naive Fibonacci is slow.

Suggested spoken answer. The function makes two recursive calls for most n, and those calls repeat many of the same smaller calculations. This creates an exponential number of calls. A loop or memoization stores previous values and reduces the work to linear time.

Practice question 3. Trace recursive digit sum for 238.

Suggested spoken answer. The function computes 8 plus sum_digits_rec(23), then 3 plus sum_digits_rec(2), then 2 plus sum_digits_rec(0). The base case returns zero, so the final sum is 2+3+8=13.

Experiment 11 - Core List Operations

Program focus: Append, insert, pop by index, remove by value, slicing, extend, indexing, concatenation, repetition, update, membership, sort/reverse, and aggregate methods.

Quick viva questions

Q1. What is the difference between append and extend?

Easy answer. append adds one object at the end; extend adds each element from an iterable.

Q2. What does insert(3,50) mean?

Easy answer. Insert 50 before the element currently at index 3.

Q3. Compare pop(4) and remove(40).

Easy answer. pop removes by index and returns the removed value; remove deletes the first matching value.

Q4. What does A[0:7] include?

Easy answer. Indices 0 through 6; the stop index 7 is excluded.

Q5. What does A+B do for lists?

Easy answer. It creates a new concatenated list; it does not mutate A or B.

Q6. What does A*4 do?

Easy answer. It repeats the sequence four times in a new list.

Q7. How does sort differ from sorted?

Easy answer. list.sort mutates the list and returns None; sorted returns a new sorted list.

Q8. What if remove(value) cannot find the value?

Easy answer. It raises ValueError.

Q9. What if an index is out of range?

Easy answer. Indexing or pop by that index raises IndexError.

Q10. What does A.copy() create?

Easy answer. A shallow copy of the outer list; nested mutable objects are still shared.

Q11. How does list comparison work?

Easy answer. Lists compare lexicographically element by element, not by length alone.

Q12. what happens to references after insertion? [General knowledge]

Easy answer. The list may shift elements internally; external iterators/references to list positions should not be treated as stable indices.

Experiment 11 - Core List Operations - expanded answers

Practice question 1. Explain index-based and value-based deletion.

Suggested spoken answer. pop(4) removes the element at index four, while remove(40) searches for the value 40 and removes its first occurrence. pop can also return the deleted value; remove returns no value and raises ValueError if the target is absent.

Practice question 2. Explain slicing and common list operations.

Suggested spoken answer. A slice A[start:stop] includes the start index and excludes the stop. append adds one item; extend adds all items from another iterable. Concatenation and repetition create new lists, while sort and reverse change the existing list.

Practice question 3. What should you say about copy and nested lists?

Suggested spoken answer. copy() makes a shallow copy. The outer list is different, but any inner lists or other mutable objects remain shared. Editing a nested object through one list can therefore be seen through the other.

Experiments 11/12 - Advanced Lists and Sets

Program focus: Reverse iteration, manual min/max, common elements, set conversion/search/difference/union/intersection.

Quick viva questions

Q1. How does reverse iteration with range work?

Easy answer. range(len(A)-1,-1,-1) starts at the last index and steps backward through index zero.

Q2. Why initialize min and max from A[0]?

Easy answer. It gives an actual list value as the starting candidate, but assumes the list is nonempty.

Q3. What is the complexity of the manual common-elements loop?

Easy answer. With list membership, roughly O(n*m) for lists of lengths n and m.

Q4. What does set(List1) do with duplicates?

Easy answer. It removes duplicate values, but set iteration order is not a reliable display order.

Q5. What are union, intersection, and difference?

Easy answer. Union combines members; intersection keeps shared members; difference keeps members in the left set but not the right.

Q6. What is the output of {1,2,3,4}.difference({3,4,5,6})?

Easy answer. The set {1,2}.

Q7. How do you create an empty set?

Easy answer. Use set(); {} creates an empty dictionary.

Q8. What if min/max code receives an empty list?

Easy answer. Accessing A[0] raises IndexError; check emptiness first.

Q9. Why can set membership be fast on average?

Easy answer. Sets use hash-based lookup, typically average O(1), though collisions and hashing costs matter.

Q10. do sets preserve insertion order? [General knowledge]

Easy answer. No; sets are unordered collections, so do not rely on their printed order.

Experiments 11/12 - Advanced Lists and Sets - expanded answers

Practice question 1. Explain the common-elements code and its cost.

Suggested spoken answer. It loops through List1 and checks whether each item is in List2. Matching values are appended to Common. Because membership in a list scans elements, the nested work can be O(n*m). If List1 contains repeated values, repeated matches can also appear.

Practice question 2. Compare a list and a set for this experiment.

Suggested spoken answer. A list preserves order and allows duplicates, while a set removes duplicates and supports mathematical operations such as union and intersection. Set output order is not guaranteed, so convert to a sorted list if a stable display order is needed.

Practice question 3. Explain the min/max loop's boundary assumption.

Suggested spoken answer. It initializes both values from A[0], which is valid only if the list has at least one element. Each later value is compared to update the current maximum and minimum. For an empty list, the code must handle that case before indexing.

Experiment 13 - Tuple Operations

Program focus: Access/slice/concatenate/sum/length/search/filter, deduplicate in order, distinct second extremes, reverse, and delete binding.

Quick viva questions

Q1. What is a tuple?

Easy answer. An ordered, immutable sequence.

Q2. How is a one-element tuple written?

Easy answer. (value,) with the comma; parentheses alone do not make it a tuple.

Q3. Can a tuple be concatenated or sliced?

Easy answer. Yes. These operations create a new tuple.

Q4. How does dict.fromkeys remove duplicates while preserving order?

Easy answer. It creates dictionary keys from the values, retaining first insertion order; tuple(...) converts the keys back to a tuple.

Q5. How does the second-smallest calculation work?

Easy answer. It sorts the distinct values and chooses index 1; second largest is index -2.

Q6. Why check there are at least two unique values?

Easy answer. Otherwise those indices do not exist.

Q7. How is tuple reversal built in the loop?

Easy answer. Each item is prepended as a one-element tuple, (i,)+Rev.

Q8. What does del A do?

Easy answer. It removes the name binding A; it does not mutate a tuple.

Q9. What is the complexity of repeated tuple prepending?

Easy answer. It repeatedly creates new tuples, so it can take O(n^2) time overall.

Q10. can a tuple contain a list? [General knowledge]

Easy answer. Yes, but then the tuple contains a mutable object and is not hashable as a dictionary key.

Experiment 13 - Tuple Operations - expanded answers

Practice question 1. Explain tuple immutability and slicing.

Suggested spoken answer. A tuple’s element references cannot be reassigned after creation. It can still be indexed, searched, concatenated, or sliced; concatenation and slicing create new tuples rather than modifying the original.

Practice question 2. Explain duplicate removal and second extremes.

Suggested spoken answer. dict.fromkeys keeps the first occurrence of each value in insertion order, and converting its keys back to a tuple preserves that order. For second-smallest and second-largest, the code sorts a set so each value appears once, then selects the second item from each end.

Practice question 3. What happens after del A?

Suggested spoken answer. The name A is removed from the current scope. The tuple object itself is not changed; if another name refers to it, that reference remains. Trying to print A afterward raises NameError, which is why that print was removed in the corrected listing.

Experiment 14 - Dictionaries

Program focus: Create/display/access/count/check/update/add/delete/popitem/clear/keys/values/aggregate/reverse items.

Quick viva questions

Q1. What is a dictionary?

Easy answer. A mutable mapping from unique, hashable keys to values.

Q2. How do you iterate over key-value pairs?

Easy answer. Use Student.items() and unpack each pair into Key and Value.

Q3. What does Key_Search in Student test?

Easy answer. It checks whether the key exists, not whether a value exists.

Q4. What happens when assigning to an existing key?

Easy answer. Its value is updated; the key is not duplicated.

Q5. What does del Student["Location"] do?

Easy answer. It removes that key and its value; a missing key raises KeyError.

Q6. What does popitem remove?

Easy answer. In modern Python, it removes and returns the last inserted pair.

Q7. What is Student.keys() or values()?

Easy answer. A dynamic view of keys or values, not a copied list.

Q8. How does reversed(list(Student.items())) work?

Easy answer. It snapshots the pairs in a list, reverses their order, then constructs a new dict.

Q9. What if a key is missing during direct access?

Easy answer. Student[key] raises KeyError; get() can supply a default.

Q10. what can be a dictionary key? [General knowledge]

Easy answer. A hashable value whose equality/hash remain stable, such as strings, numbers, or tuples of hashable values.

Experiment 14 - Dictionaries - expanded answers

Practice question 1. Explain key lookup, update, and deletion.

Suggested spoken answer. A dictionary stores unique keys with associated values. The in operator checks key presence. Assigning Student["Age"]=20 updates that entry; assigning a new key adds an entry. del removes a key and raises KeyError if it is absent.

Practice question 2. Explain popitem and reverse order.

Suggested spoken answer. popitem removes and returns the most recently inserted pair in modern Python. The reverse example first converts the dictionary items to a list, reverses that list, then builds a new dictionary. Dictionaries preserve insertion order in current Python.

Practice question 3. How do numeric dictionary values get summarized?

Suggested spoken answer. Marks.values() gives a view of the stored values. sum, max, and min operate on those values, not on the subject-name keys. These operations assume the dictionary is nonempty for max and min.

Experiment 15 - Strings and Text Processing

Program focus: Includes one list slice mislabeled as string slicing, string methods, character classes, frequencies, word operations, duplicate removal, reversal, and basic email validation.

Quick viva questions

Q1. Are Python strings mutable?

Easy answer. No. String methods such as replace return a new string; assigning it back rebinds the variable.

Q2. What does split() do without an argument?

Easy answer. It splits on runs of whitespace and avoids empty tokens from repeated spaces.

Q3. Why is Words[0] safe after split()?

Easy answer. split() returns no empty word tokens, but the code should still handle an empty sentence before indexing.

Q4. What does isalpha classify?

Easy answer. Whether a character is alphabetic according to Unicode, not only English A-Z.

Q5. What does isdigit classify?

Easy answer. Unicode digit characters; it is not limited strictly to ASCII 0-9.

Q6. Why can the most-frequent-character result vary on a tie?

Easy answer. The max searches a set of characters, whose iteration order is not a stable tie-break rule.

Q7. What is the complexity of max(set(Text), key=Text.count)?

Easy answer. It can be O(n^2), because Text.count scans the whole string for each distinct character.

Q8. Why is the email validation only basic?

Easy answer. It checks for simple @ and dot conditions but not the full email syntax or domain rules.

Q9. What does the word deduplication code preserve?

Easy answer. It preserves first occurrence order and is case-sensitive; the duplicate-detection example lowercases first.

Q10. What is incorrect about the first “string operations” listing?

Easy answer. It slices a list named Numbers; it is not a string example.

Q11. why use regex or a validation library for emails? [General knowledge]

Easy answer. Email syntax has many edge cases, so a simple substring test is only a rough screen.

Experiment 15 - Strings and Text Processing - expanded answers

Practice question 1. Explain the character-counting loop.

Suggested spoken answer. For each character, it first checks whether the character is alphabetic. If so, it lowercases it and checks whether it is one of the vowels; otherwise it counts it as a consonant. Digits go to the digit counter, and punctuation or spaces count as special in this program.

Practice question 2. Explain duplicate-word removal.

Suggested spoken answer. The code splits the sentence into words, then uses dict.fromkeys to keep each word's first occurrence and insertion order. It joins the resulting list with spaces. This version is case-sensitive, so Python and python are different tokens.

Practice question 3. Why call the email checker "basic"?

Suggested spoken answer. It checks simple conditions such as exactly one @ and the presence of a dot, but it does not validate all legal local-part or domain formats. It is suitable only as a classroom example, not a complete email validator.

Experiment 16 - File Operations

Program focus: Checks existence, reads first/last lines, counts content, replaces text, finds extreme lines/common word, compares files, and combines files.

Quick viva questions

Q1. Why use with open(...) as file?

Easy answer. The context manager closes the file automatically when the block finishes, including when an exception occurs.

Q2. What does os.path.exists check?

Easy answer. Whether a filesystem path currently exists; it does not guarantee the path can be opened as a readable file.

Q3. What does readline return at end-of-file?

Easy answer. An empty string, which the first-three-lines loop uses to stop early.

Q4. How does lines[-3:] behave with fewer than three lines?

Easy answer. It returns all available lines; it does not raise an error.

Q5. How does splitlines differ from split?

Easy answer. splitlines counts line boundaries; split with no separator divides on whitespace, useful for word counting.

Q6. What does str.replace do in the file example?

Easy answer. It replaces all matching non-overlapping occurrences in the in-memory string and returns a new string.

Q7. Why is opening with "w" important?

Easy answer. It truncates the old file before writing the modified contents.

Q8. How does most_common(1) behave?

Easy answer. Counter returns a list containing the most common (word,count) pair, or an empty list for no words.

Q9. How are combined files separated?

Easy answer. The code writes each whole file and then an extra newline.

Q10. What file errors are not handled?

Easy answer. Missing files, permission errors, decoding issues, or write failures can raise exceptions.

Q11. why specify encoding="utf-8"? [General knowledge]

Easy answer. It makes text decoding/encoding behavior more consistent across machines.

Experiment 16 - File Operations - expanded answers

Practice question 1. Explain the replace-word file program.

Suggested spoken answer. It opens the file and reads all text into memory. It replaces every exact occurrence of the old text with the new text, then opens the same file in write mode and writes the updated contents. Write mode truncates the previous contents, so the full modified text must be written back.

Practice question 2. Compare first-three-lines and last-three-lines reading.

Suggested spoken answer. The first-lines program calls readline only a few times, so it can stop early without loading the whole file. The last-lines example uses readlines and stores every line, then slices the last three. That is simpler, but uses memory proportional to the file size.

Practice question 3. What happens when comparing two files?

Suggested spoken answer. The code reads both complete files as text strings and compares them exactly, including whitespace and line endings as represented by text mode. A difference in one character makes the strings unequal.

Experiment 4 - NumPy Linear Algebra

Program focus: Vector/matrix arithmetic, transpose, complex conjugate transpose, determinant, inverse, and eigenvalues/eigenvectors for A and B.

Quick viva questions

Q1. What is a NumPy array?

Easy answer. A typed multidimensional array supporting vectorized numeric operations.

Q2. What does v1+v2 do for equal-shaped vectors?

Easy answer. Elementwise addition; [1,2,3]+[4,5,6]=[5,7,9].

Q3. What is the difference between A*B and matrix multiplication?

Easy answer. For NumPy arrays, * is elementwise; @ or np.dot performs matrix multiplication.

Q4. What does A.T do?

Easy answer. It swaps rows and columns; for real matrices it is the transpose.

Q5. What is conjugate transpose?

Easy answer. Transpose the matrix and take the complex conjugate of every element; NumPy uses C.conj().T.

Q6. What is det(A) for [[1,2],[3,4]]?

Easy answer. 1*4-2*3=-2.

Q7. What is the inverse of A?

Easy answer. [[-2,1],[1.5,-0.5]], because multiplying A by it gives the identity.

Q8. What defines an eigenvalue/eigenvector?

Easy answer. A nonzero vector v is an eigenvector if A v=lambda v for scalar lambda.

Q9. What happens if a matrix is singular?

Easy answer. It has no inverse; np.linalg.inv raises LinAlgError if singularity is detected.

Q10. Why do printed floating results have small rounding errors?

Easy answer. Floating-point calculations use finite precision; compare numerically with tolerances where appropriate.

Q11. what does determinant zero imply? [General knowledge]

Easy answer. The square matrix is singular and its columns are linearly dependent.

Experiment 4 - NumPy Linear Algebra - expanded answers

Practice question 1. Explain the matrix operations in the experiment.

Suggested spoken answer. A and B are 2-by-2 arrays. Addition and subtraction combine matching entries; A.T swaps rows and columns. The complex example uses conj().T to both transpose and conjugate. The determinant and inverse describe further properties of A.

Practice question 2. Explain determinant and inverse of A.

Suggested spoken answer. For A=[[1,2],[3,4]], the determinant is 1*4 minus 2*3, which equals -2. Because it is nonzero, A is invertible. Its inverse is [[-2,1],[1.5,-0.5]], and A multiplied by that inverse is the identity, up to floating-point precision.

Practice question 3. What does eig return?

Suggested spoken answer. np.linalg.eig returns eigenvalues and corresponding right eigenvectors. Each eigenvector column v satisfies A@v=lambda*v for its eigenvalue. The values are not necessarily sorted, and numerical output may include small rounding differences.

Experiment 5 - Quantum-State Vectors

Program focus: Complex NumPy vectors, norm, normalization, vdot inner product, outer product, orthogonality, and computational basis vectors.

Quick viva questions

Q1. What is a vector norm?

Easy answer. A measure of vector magnitude; default np.linalg.norm for a one-dimensional vector is its Euclidean norm.

Q2. Why normalize v2=[1,1]?

Easy answer. Its norm is sqrt(2), so division gives [1/sqrt(2),1/sqrt(2)] with norm one.

Q3. What does np.vdot do for complex vectors?

Easy answer. It conjugates the first argument and computes the inner product; for vectors it flattens inputs.

Q4. What is the inner product of [1,0] and [1/sqrt2,1/sqrt2]?

Easy answer. 1/sqrt2.

Q5. What does np.outer(a,b) produce?

Easy answer. A matrix whose entry (i,j) is a[i]*b[j]; this source conjugates v2_norm before passing it as the second vector.

Q6. How do you test orthogonality?

Easy answer. Their inner product is zero; the code uses np.isclose to account for floating-point tolerance.

Q7. Why are |0> and |1> orthogonal?

Easy answer. [1,0] dot [0,1] is zero.

Q8. Why use dtype=complex?

Easy answer. It permits complex amplitudes without losing imaginary parts.

Q9. what condition makes a vector a valid pure quantum state? [General knowledge]

Easy answer. Its norm must be one, so the squared magnitudes of its amplitudes sum to one.

Experiment 5 - Quantum-State Vectors - expanded answers

Practice question 1. Explain the normalization and inner-product steps.

Suggested spoken answer. v1 is [1,0], which already has norm one. v2 is [1,1], whose norm is sqrt(2), so dividing by that norm makes a unit vector. vdot conjugates the first input and computes their inner product; the result with v1 is 1/sqrt(2).

Practice question 2. Explain orthogonality in the code.

Suggested spoken answer. The basis vectors |0>=[1,0] and |1>=[0,1] have zero inner product, so they are orthogonal. The code checks with np.isclose rather than exact equality because floating-point operations can introduce tiny numerical errors.

Practice question 3. How is the outer product different from the inner product?

Suggested spoken answer. An inner product returns one scalar. An outer product creates a matrix from every pair of components. The source explicitly conjugates the second vector before using np.outer, which is a code choice distinct from np.vdot's automatic conjugation of its first argument.

Experiment 6 - Quantum-State Transformations

Program focus: Builds |0>, applies a Hadamard matrix, multiplies X@H, and applies the combined matrix to the initial column vector.

Quick viva questions

Q1. What is the initial state?

Easy answer. [[1],[0]], the column-vector representation of |0>.

Q2. What does H do to |0>?

Easy answer. It creates |+>=(|0>+|1>)/sqrt(2).

Q3. What is the Pauli-X matrix?

Easy answer. [[0,1],[1,0]], which exchanges the |0> and |1> amplitudes.

Q4. What does X@H mean in application order?

Easy answer. Matrix multiplication acts right to left on a state: H acts first, then X.

Q5. What is XH|0>?

Easy answer. X leaves |+> unchanged, so it remains [1/sqrt2,1/sqrt2].

Q6. Why use np.dot?

Easy answer. It performs matrix multiplication for these compatible arrays; @ is the equivalent operator syntax.

Q7. Why is a matrix applied by left multiplication?

Easy answer. The state is represented as a column vector and the gate matrix multiplies it from the left.

Q8. What is a common shape error?

Easy answer. Multiplying incompatible matrix/vector dimensions; here each gate is 2x2 and the state is 2x1.

Q9. what makes a quantum gate unitary? [General knowledge]

Easy answer. Its conjugate transpose times itself equals the identity, preserving state norm.

Experiment 6 - Quantum-State Transformations - expanded answers

Practice question 1. Trace the state transformation.

Suggested spoken answer. The initial column vector is |0>=[1,0]. H multiplies it into [1/sqrt2,1/sqrt2], the equal superposition. The combined matrix is X@H, so H acts first and then X. Since |+> is unchanged by X, the final state remains [1/sqrt2,1/sqrt2].

Practice question 2. Why does multiplication order matter?

Suggested spoken answer. Matrix multiplication is not generally commutative, so X@H and H@X can represent different operations. When applying a product to a column state, the rightmost matrix acts first. The code correctly computes X times H times the initial state.

Practice question 3. What property should a valid gate preserve?

Suggested spoken answer. A quantum gate should be unitary. Unitary operations preserve the norm of the state, which keeps total probability equal to one.

Experiment 7 - Identity, Kronecker Product, and Unitarity

Program focus: Builds I1/I2/I3 with eye and kron, verifies state unchanged with allclose, and checks X†X=I.

Quick viva questions

Q1. What does np.eye create?

Easy answer. A two-dimensional identity array with ones on the main diagonal and zeros elsewhere.

Q2. What is np.kron?

Easy answer. The Kronecker product; it combines matrices into a larger block matrix.

Q3. What dimensions are I1, I2, and I3?

Easy answer. 2x2, 4x4, and 8x8, corresponding to one, two, and three qubits.

Q4. Why normalize psi_2 and psi_3?

Easy answer. They are intended as quantum statevectors, so their norms should be one.

Q5. Why use np.allclose instead of ==?

Easy answer. Floating-point results can differ by tiny rounding amounts; allclose applies a tolerance.

Q6. How does identity act on a state?

Easy answer. It leaves every component unchanged.

Q7. What does X.conj().T compute?

Easy answer. The conjugate transpose X-dagger; X is real, so it equals X transpose.

Q8. How does X†X test unitarity?

Easy answer. A matrix U is unitary when U†U equals identity.

Q9. What is XI=np.kron(X,I1)?

Easy answer. A four-by-four two-qubit operator applying an X factor on one tensor component and identity on the other, under the chosen basis convention.

Q10. why is basis order important in multi-qubit Kronecker products? [General knowledge]

Easy answer. Changing tensor-factor order changes which subsystem a matrix acts on.

Experiment 7 - Identity, Kronecker Product, and Unitarity - expanded answers

Practice question 1. Explain how the identity matrices are built.

Suggested spoken answer. I1 is a 2-by-2 identity matrix. I2 is the Kronecker product of two I1 matrices, making a 4-by-4 identity, and I3 adds another factor to make an 8-by-8 identity. They represent identity operations on one, two, and three qubits.

Practice question 2. Explain the two numerical checks.

Suggested spoken answer. First, the experiment multiplies normalized statevectors by identity matrices and uses allclose to confirm that the output matches the input within numerical tolerance. Second, it calculates X conjugate-transpose times X; because X is real and its own inverse, the product is I, confirming unitarity.

Practice question 3. What is the Kronecker product used for?

Suggested spoken answer. It builds a larger matrix from smaller matrices, especially to express operations on multi-part systems. The order of factors matters because it specifies which tensor subsystem receives each operation.

Rapid Revision

Topic	Remember
Python basics	input returns str; / gives float; ** is power; tuple assignment swaps.
Conditions	Test leap years with divisible by 400 OR divisible by 4 and not 100.
Loops	%10 extracts final digit; //10 removes it; factorial starts at 1.
Functions	return sends values back; *args tuple; **kwargs dict.
Recursion	Base case stops calls; naive Fibonacci repeats work.
Lists	Mutable sequence; append one; extend iterable; slice stop excluded.
Sets	Unique, unordered values; union/intersection/difference.
Tuples	Ordered and immutable; one element uses trailing comma.
Dictionaries	Unique hashable keys; insertion order; missing [] key raises KeyError.
Strings	Immutable Unicode sequence; replace returns a new string.
Files	with open closes automatically; w overwrites; readlines loads all lines.
NumPy linear algebra	@ is matrix multiply; .T transpose; conj().T adjoint.
Quantum vectors	Normalize to norm 1; vdot conjugates first argument; orthogonal inner product is zero.
Quantum gates	H creates |+>; X@H acts H first on a column vector.
Unitary check	U†U=I; np.allclose allows rounding tolerance.

Mock Viva

Answer aloud, then compare with each checkpoint.

1. Why does input need conversion before arithmetic?

Checkpoint. input() returns text; int() or float() converts it.

2. Why is 2000 a leap year but 1900 is not?

Checkpoint. Century years must be divisible by 400.

3. What does 153 become when reversed?

Checkpoint. 351.

4. What are the base cases for factorial_rec?

Checkpoint. 0 and 1 both return 1.

5. How does *args differ from **kwargs?

Checkpoint. Extra positional tuple versus named-argument dictionary.

6. What does A[0:7] exclude?

Checkpoint. Index 7.

7. Why might set output order vary?

Checkpoint. Sets are unordered.

8. Why is a tuple safe as a dict key only sometimes?

Checkpoint. Its contents must be hashable/immutable.

9. Why can the most frequent character tie be arbitrary?

Checkpoint. The search iterates through a set.

10. What does file open mode w do?

Checkpoint. Truncates/replaces the existing file contents.

11. What is determinant of [[1,2],[3,4]]?

Checkpoint. -2.

12. Why is vdot used for complex vectors?

Checkpoint. It conjugates the first argument.

13. What is X@H|0>?

Checkpoint. |+>, since H acts first and X leaves |+> unchanged.

14. How test unitarity numerically?

Checkpoint. Compute U.conj().T@U and compare to identity with allclose.

Viva answer checklist

Define the concept; trace the actual code; give one edge case; explain relevant type/shape; state complexity for algorithms; distinguish mathematical output from printed formatting.

References and source notes

Primary source: supplied cqp.md, which compiles code from 15 PDFs. It notes corrections to leap-year/tie logic, negative factorial and digit handling, tuple access and duplicate handling, string empty cases, and file examples. The guide follows those corrected listings.

Python data structures: lists, sets, tuples, dictionaries
https://docs.python.org/3/tutorial/datastructures.html

Python built-in types: string behavior
https://docs.python.org/3/library/stdtypes.html

NumPy vdot
https://numpy.org/doc/2.0/reference/generated/numpy.vdot.html

NumPy Kronecker product
https://numpy.org/doc/stable/reference/generated/numpy.kron.html

NumPy matrix inverse
https://numpy.org/doc/stable/reference/generated/numpy.linalg.inv.html

NumPy eigenvalues and eigenvectors
https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html

General-knowledge extensions are labeled. NumPy matrix calculations use floating-point arithmetic, so numerical checks use tolerances.